Méthodologie de conception

DSI3.1
DSI3.2

Processus Unifié

N.SASSI & L.BEN RHOUMA

2024-2025

Plan du cours

- Introduction aux approches méthodologiques
- Méthodes Agiles
- Scrum
- Processus Unifié

Eléments du chapitre

- Introduction
- PU (Définition, objectifs, principes)
- Artefacts d’un PU
- Caractéristiques du Processus Unifié
- Les phases du Processus Unifié
- Les activités de processus
- Les 4 « P » du développement logiciel

Introduction

L'objectif sous-jacent que poursuit une entreprise n'est évidement pas de posséder un bon système informatique, mais d'exploiter les processus métiers (ou les systèmes intégrés) pour répondre au mieux à la demande du marché et accélérer la production de biens et de services de qualité à un prix raisonnable.

Introduction

- La création des logiciels de plus en plus imposants et complexes.
- On cherche des logiciels:
  - plus adoptés à nos besoins,
  - répondent aux exigences du marché,
  - dans des délais fixes,
  - avec des coûts réduits,
  - mieux documentés.

Processus

- Un processus définit une séquence d’étapes, en partie ordonnées, qui concourent à l’obtention d’un système logiciel ou à l’évolution d’un système existant.

- L’objet d’un processus de développement est de produire des logiciels de qualité qui répondent aux besoins de leurs utilisateurs dans des temps et des coûts prévisibles.

Processus Unifié

« Le processus unifié est un processus de développement logiciel, c’est-à-dire qu’il regroupe les activités à mener pour transformer les besoins d’un utilisateur en système logiciel » (Jacobson, Booch, Rumbaugh 1999)

Le processus unifié (PU) prend en charge le cycle de vie du logiciel et notamment son développement, pour les logiciels orientés objets.

- Est itératif et incrémental
- Est une méthode de développement associée à l’utilisation du langage de modélisation UML.

Processus Unifié

Le résultat de la fusion des méthodes Objectory d'Ivar Jacobson, Booch de Grady Boochet OMT de James Rumbaugh, enrichi de nombreux apports issus des travaux d'élaboration du standard UML et du produit commercial RUP (Rational Unified Process).

Processus Unifié

- Le Processus Unifié ou UP (Unified Process) est une méthode générique de développement de logiciel développée par les concepteurs d'UML.

- Générique signifie qu'il est nécessaire d'adapter UP au contexte du projet, de l'équipe, du domaine et/ou de l'organisation.

- Il existe donc un certain nombre de méthodes issues de UP comme par exemple RUP (Rational Unified Process), 2TUP (Two Track Unified Process) …

Objectifs

- L'objectif d'un processus unifié est de maîtriser la complexité des projets informatiques en diminuant les risques.

- UP répond aux préoccupations suivantes :
  - **QUI** participe au projet ?
  - **QUOI**, qu'est-ce qui est produit durant le projet ?
  - **COMMENT** doit-il être réalisé ?
  - **QUAND** est réalisé chaque livrable ?

Objectifs

- Produire un logiciel de **qualité** en respectant des **contraintes** de **délai**, de **coûts** et de **performance**
- Fournir des lignes directrices pour un **développement efficace** d'un logiciel de **qualité**, en **réduisant les risques** et en **améliorant les prévisions**
- Décrire les **meilleures méthodes de travail** pour apprendre des expériences précédentes et l'amélioration du support de formation
- Établir une vision et une culture commune

Variantes du UP

- **RUP : Rational Unified Process :** Instanciation par Rational Software (IBM) des préceptes UP
- **EUP : Enterprise Unified Process :** Instanciation intégrant les phases de post-implantation et décrivant le cycle de vie du logiciel.
- **XUP : Extreme Unified Process :** Instanciation hybride intégrant UP avec Extreme Programming.
- **AUP : Agile Unified Process:** partie des préceptes UP permettant l'agilité du développement, instanciation partielle de la méthode mettant l'accent sur l'optimisation et l'efficience sur le terrain plus que sur le modèle théorique.

- **2TUP : Two Tracks Unified Process :** Instanciation de UP proposé par Valtech prenant en compte les aléas et contraintes liées aux changements perpétuels et rapides des SI des entreprises.
- **EssUP : Essential unified process** : (Ivar Jacobson) propose une mouture de UP integrant certains concepts de la méthodes Agile et de ProcessImprovement (intégration prévue aux outils de travail collaboratifs "Visual Studio Team System" de Microsoft).

Principes

Il n'existe pas un seul processus de développement, ni de processus standard

• CEPENDANT des caractéristiques essentielles peuvent être mises en avant :
– Pilotage par les cas d'utilisation
– Focalisation sur l'architecture
– Utilisation de « patrons » de conception (Design Patterns)
- Déroulement itératif et incrémental

• Accent mis sur les phases plus que sur les activités d'analyse, conception, …

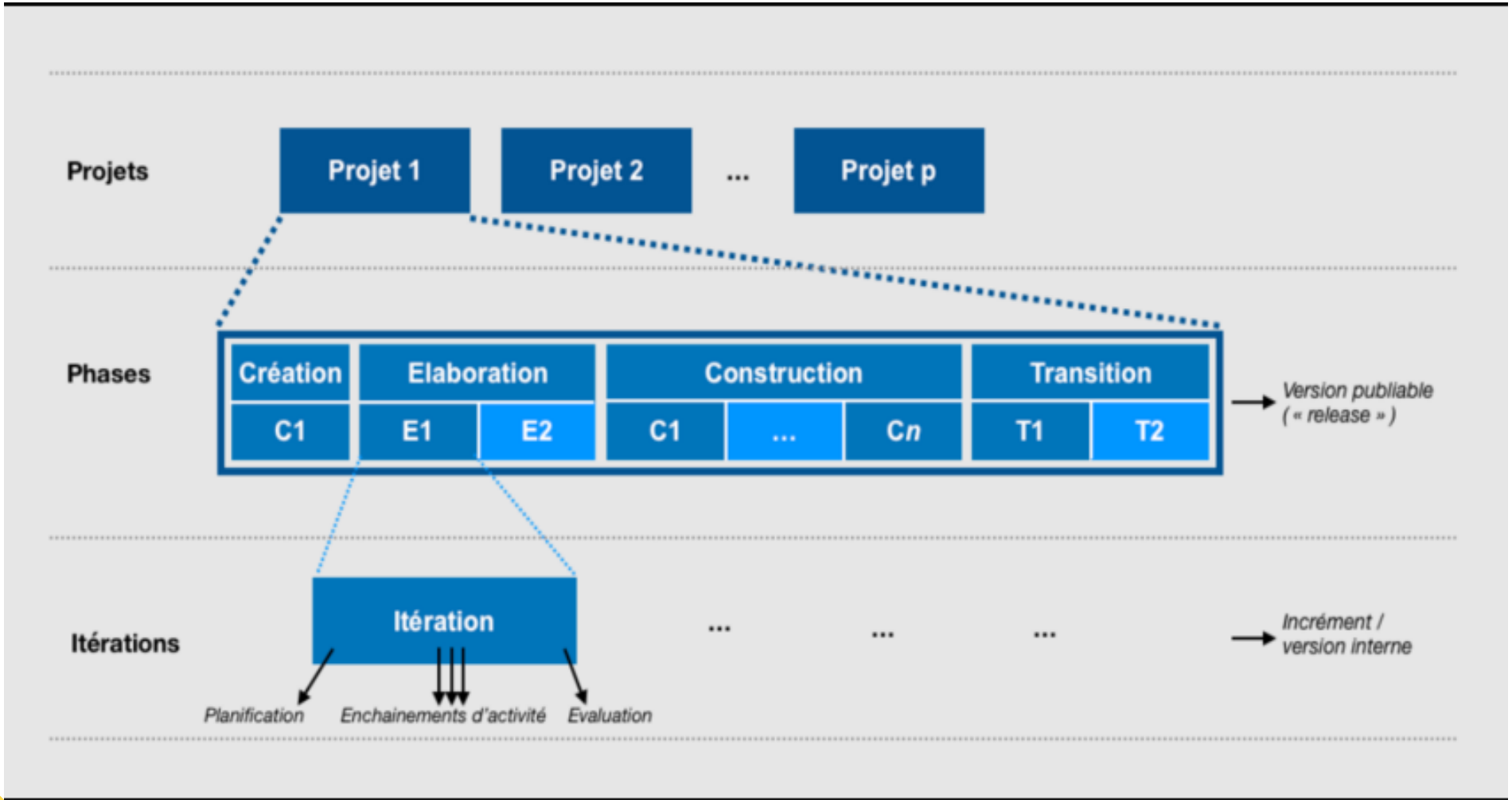
Principes

- Le PU utilise le langage UML pour la création des plans d'élaboration du logiciel.
- Le PU ne fait pas partie intégrante du standard UML.
- Utilisé pour le développement orienté objet de logiciel.

Le PU met en oeuvre les principes suivants :

- Processus **guidé par les cas d'utilisation**,
- Processus **itératif et incrémental**,
- Processus **centré sur l'architecture**,
- Processus orienté par la **réduction des risques**.

Principes


Projets
Projet 1
Projet 2
...
Projet p
Phases
Création
Elaboration
Construction
Transition
C1
E1
E2
C1
...
Cn
T1
T2
Version publiable
(« release »)
Itérations
Itération
...
...
...
Incrément /
version interne
Planification
Enchainements d'activité
Evaluation

Principes

- Considérer un produit logiciel quelconque par rapport à ses versions
- un cycle produit/ une version
- Gérer chaque cycle de développement comme un projet ayant quatre phases vue gestionnaire (manager)
- chaque phase se termine par un point de contrôle (ou jalon) permettant aux chefs de projet de prendre des décisions

Artefacts d’un projet

Tout élément d'information utilisé ou généré pendant un cycle de développement d'un système logiciel:

- morceau de code
- commentaire
- spécification statique d'une classe
- spécification comportementale d'une classe
- jeu de test, programme de test, interview d'un utilisateur potentiel du système
- description du contexte d'installation matériel
- diagramme d'une architecture globale
- Prototype
- rapport de réalisation
- modèle de dialogue,
- manuel utilisateur…

Avantages d’un processus incrémental et itératif

- Gestion de la complexité
  - pas tout en même temps, étalement des décisions importantes
- Maîtrise des risques élevés précoce
  - diminution de l’échec
  - architecture mise à l’épreuve rapidement (prototype réel)
- Intégration continue
  - progrès immédiatement visibles
  - maintien de l’intérêt des équipes (court terme, prototypes vs documents)
- Prise en compte des modifications de besoins
  - feedback, implication des utilisateurs et adaptation précoce
- Apprentissage rapide de la méthode
  - amélioration de la productivité et de la qualité du logiciel
- Adaptation de la méthode
  - possibilité d'explorer méthodiquement les leçons tirées d'une itération (élément à conserver, problèmes, éléments à essayer…)
- Mais gestion de projet plus complexe : planification adaptative

Caractéristiques: piloté par CU UML

- L’orientation forte donnée ici par UP est de montrer que le système à construire se définit d’abord avec les utilisateurs. Les **cas d’utilisation** permettent d’exprimer les interactions du système avec les utilisateurs, donc de capturer les besoins.

- Une seconde orientation est de montrer comment les cas d’utilisation constituent un vecteur structurant pour le développement et les tests du système. Ainsi le développement peut se décomposer par cas d’utilisation et la réception du logiciel sera elle aussi articulée par cas d’utilisation.

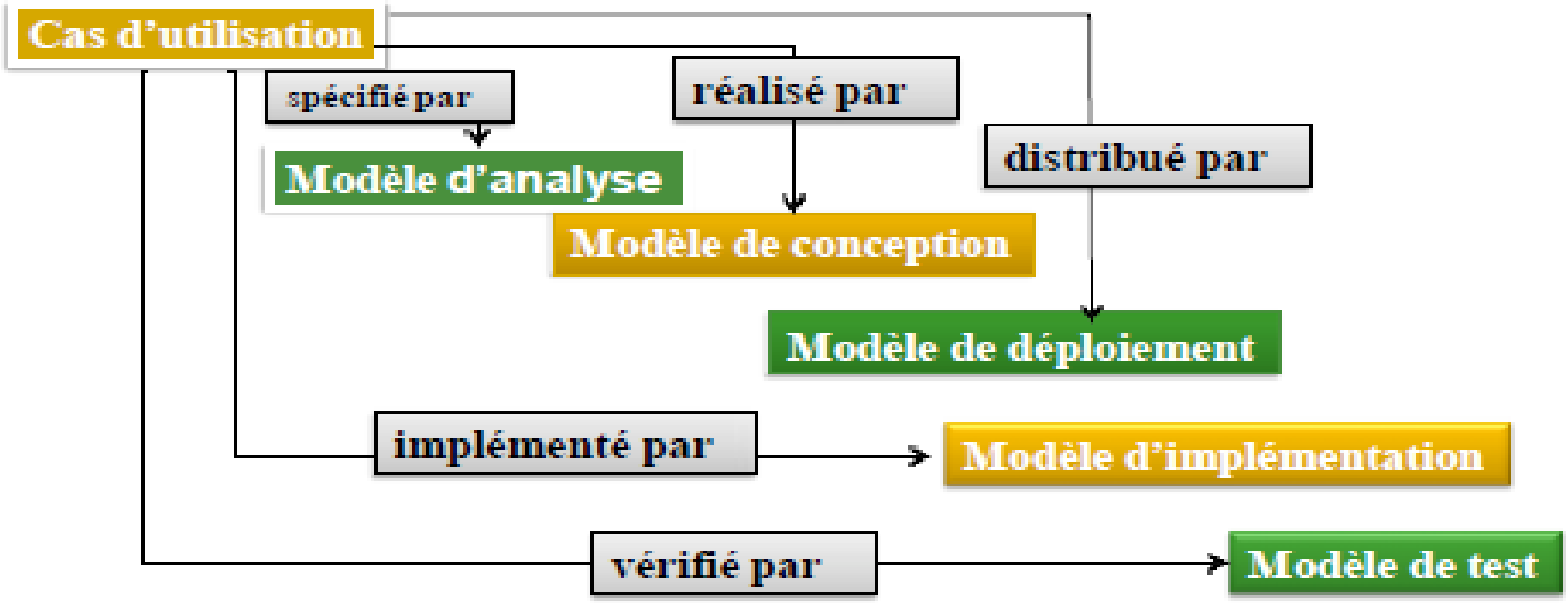
Caractéristiques: piloté par CU UML

Le but principal d'un système d'informatique est de satisfaire les besoins de client. Le processus de développement sera donc accès sur l'utilisateur. Les cas d'utilisation permettent d'illustrer ces besoins. Ils détectent puis décrivent les besoins fonctionnels et leur ensemble constitue le modèle de cas d'utilisation qui dicte les fonctionnalités complètes du système.

- Un outil de spécification des besoins du système.
- Un outil pour guider le processus de développement.
- Les cas d'utilisation sont:
- détaillés par l'analyse
  - réalisés par la conception
  - concrètement implémentés par les développeurs
  - vérifiés par les scénarios de test

Caractéristiques: piloté par CU UML

- Support de communication entre utilisateurs et concepteurs.
- Assurent la traçabilité de toute décision de conception.
- Fournissent une vision commune aux participants du projet.

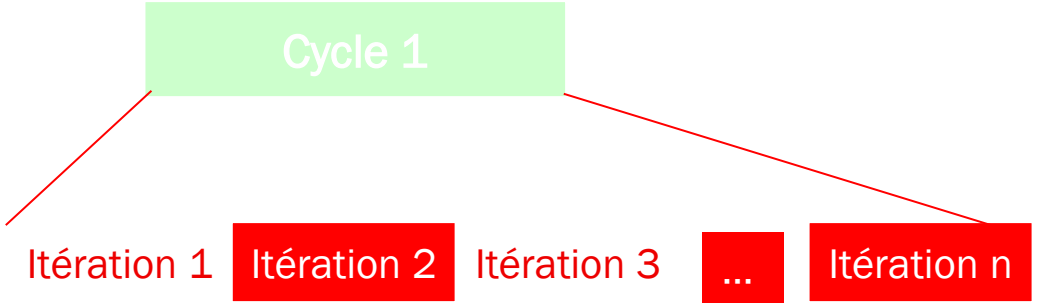


Caractèristiques: centré sur l'architecture

- Une architecture décrit les différentes vues des modèles du système à construire.
- Une architecture représente les aspects dynamique et statique du système.
- Une architecture est un lien pour les membres du projet:
- Une architecture émerge des besoins exprimés à travers les cas d'utilisation.
- Elle complète les cas d'utilisation.

- Dans un premier temps, choisir une architecture qui définit les parties générales de l'application.
- Ceci mène à la construction d'une version préliminaire du produit.
- Dans un deuxième temps, stabiliser l'architecture autour des fonctions essentielles.
- Ceci mène à la construction d'une architecture référence.
- Dans un troisième temps, l'architecture continue à se stabiliser dans les itérations suivantes.

Caractéristiques: itératif et incrémental



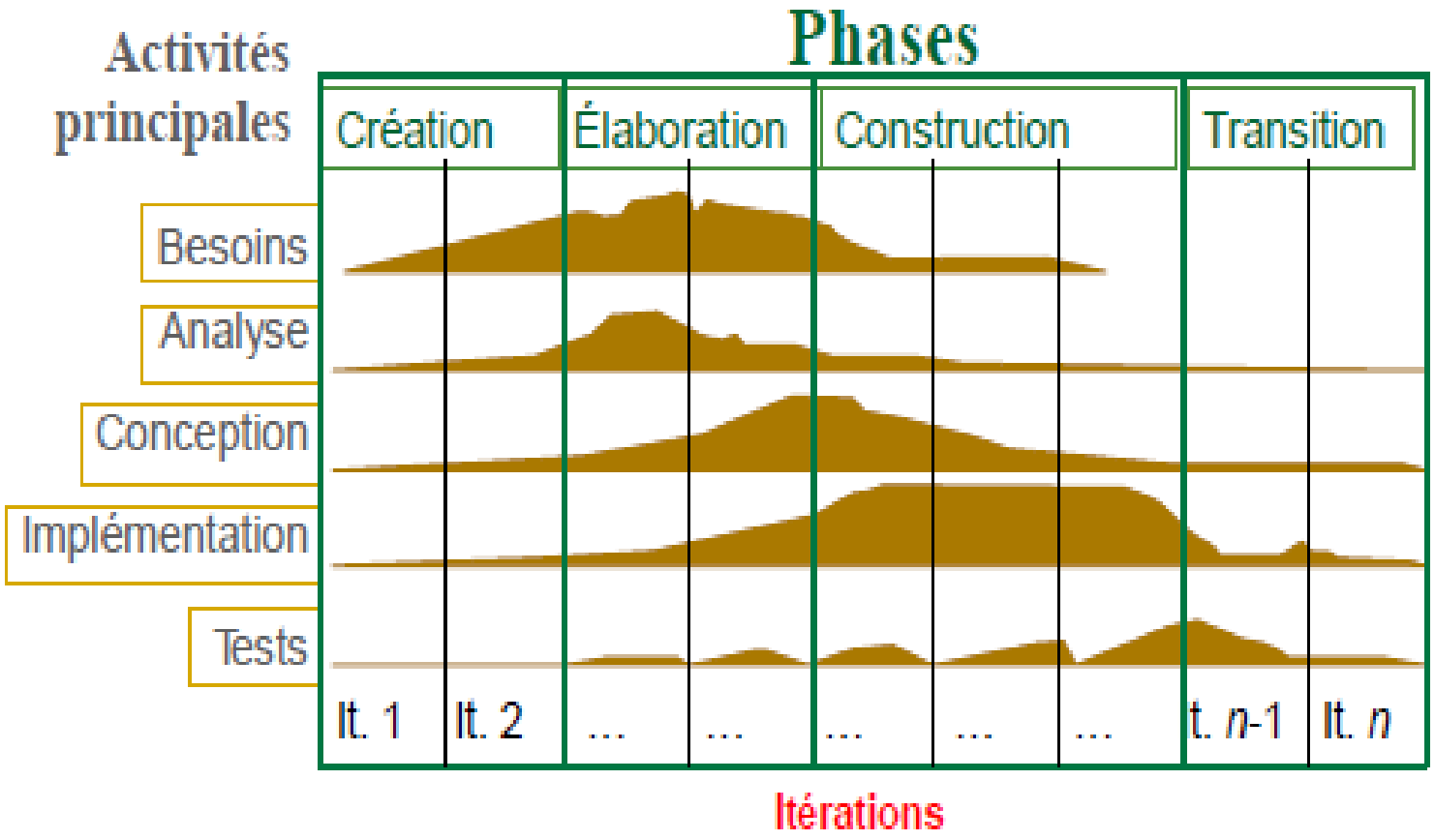
Découper le travail en plusieurs parties qui présentent des mini-projets.

- Une itération contient toutes les activités, conduit au développement d'un certain nombre de UC.
- Les itérations donnent lieu à un incrément.
- Un incrément correspond à une avancée dans les différents stades de développement.

- Le projet est découpé en itérations ou étapes de courte durée qui permettent de mieux suivre l'avancement global.
- A la fin de chaque itération une partie exécutable du système finale est produite, de façon incrémentale (par ajout).

Caractéristiques: itératif et incrémental

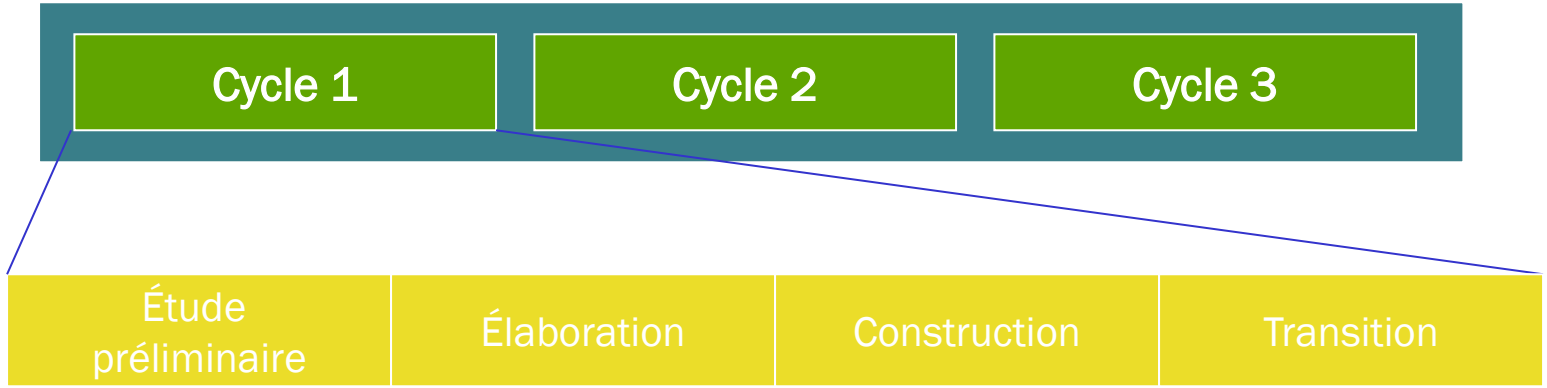
- Gestion de la complexité.
- Prise en compte des modifications de besoins.
- Apprentissage rapide de la méthode.
- Maîtrise précoce des risques.



Processus orienté par la réduction des risques

- L’analyse des **risques** doit être présente à tous les stades de développement d’un système.
- Il est important de bien évaluer les risques des développements afin d’aider à la bonne prise de décision.
- Du fait de l’application du processus itératif, UP contribue à la diminution des risques au fur et à mesure du déroulement des itérations successives.

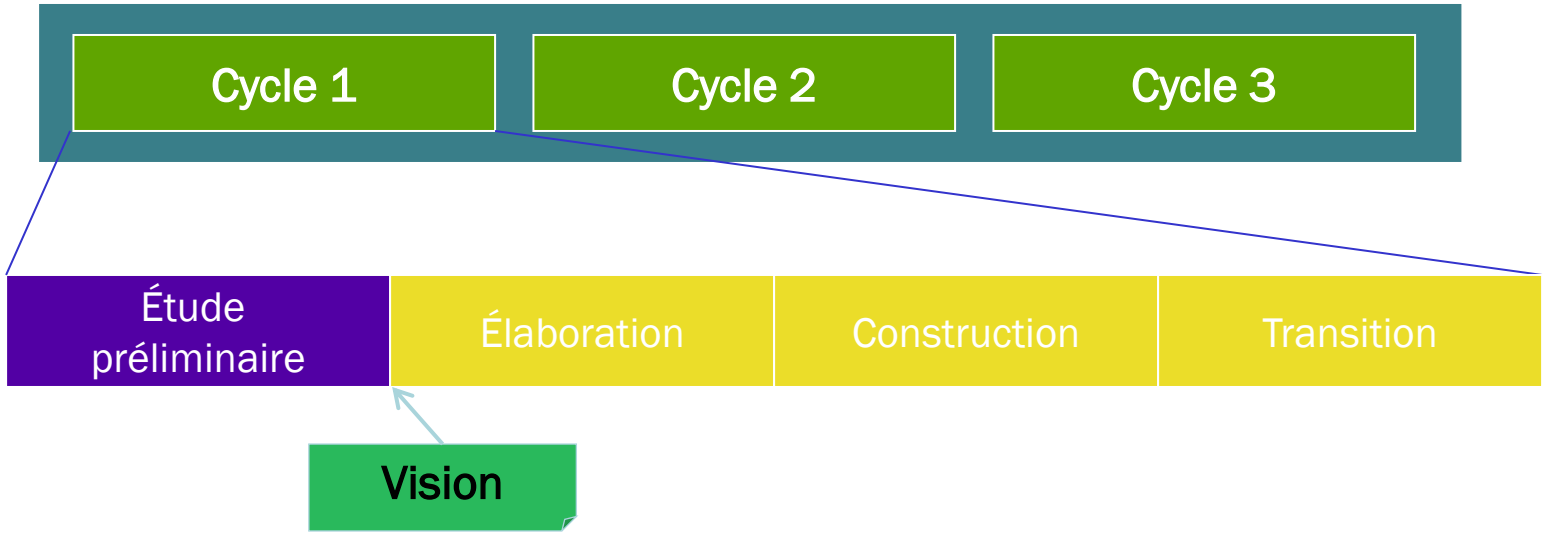
Phases
Aspect dynamique



Les phases du PU

- Le PU répète une série de cycles constituant la vie du système.
- Un cycle conduit à la livraison d'une version du produit.
- Chaque phase s'achève par un jalon définit par un ensemble d'artefacts.
- Chaque cycle de développement du PU est géré comme un projet ayant 4 phases:
  - Étude préliminaire
  - Élaboration
  - Construction
  - Transition

Phase N°1 : Etude préliminaire



- Cette phase traduit l'idée en une vision du produit fini.
- Elle répond aux questions suivantes:
- Que va faire le système pour ses utilisateurs?
- A quoi peut rassembler l'architecture du système?
- Quels sont l'organisation et les coûts de ce produit?
- Jalon
- « vision du projet » = document
- Accepter le projet ?

Phase N°1 : Etude préliminaire

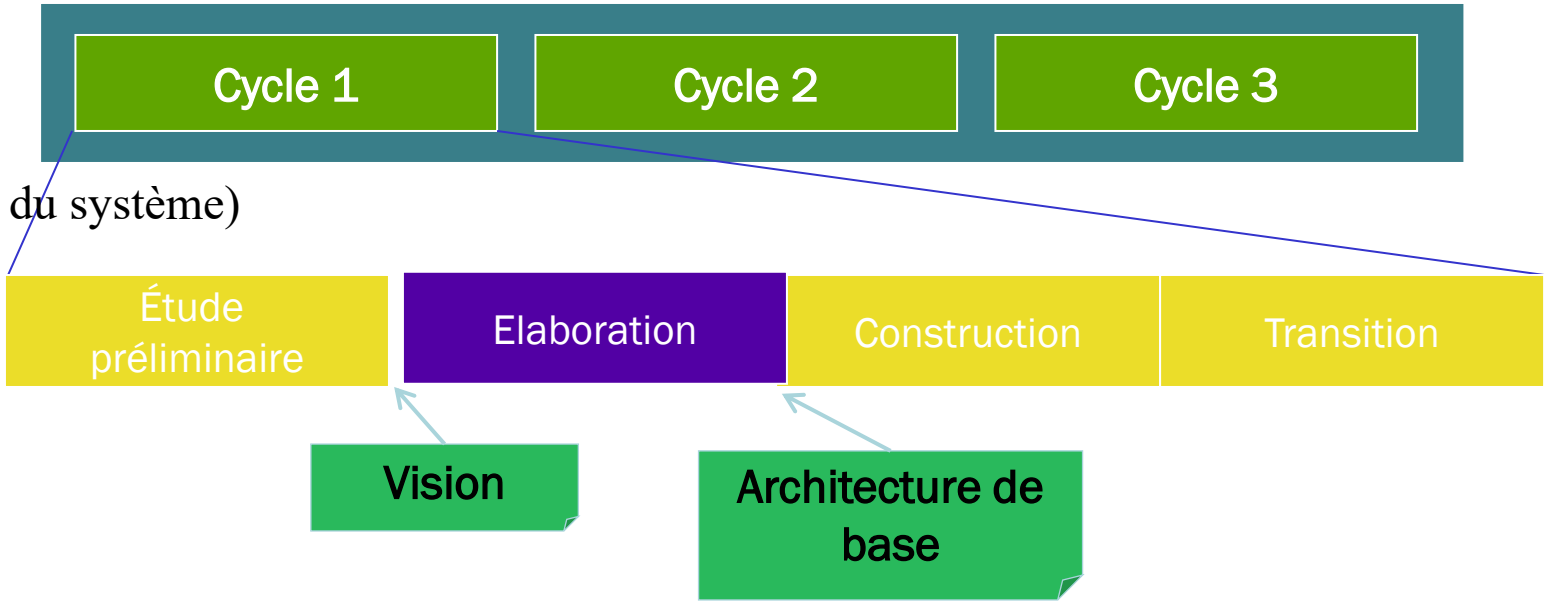
- Cette phase correspond à **l'initialisation du projet** où l'on mène une étude d'opportunité et de faisabilité du système à construire. Une évaluation des risques est aussi réalisée dès cette phase.

- En outre, une identification des principaux cas d'utilisation accompagnée d'une description générale est modélisée dans un diagramme de cas d'utilisation afin de définir le périmètre du projet. Il est possible, à ce stade, de faire réaliser des maquettes sur un sous-ensemble des cas d'utilisation identifiés.

- Ce n'est qu'à l'issue de cette première phase que l'on peut considérer le projet véritablement lancé.

Phase N°2 : Elaboration

- Spécifier la plupart des cas d'utilisation
- Concevoir l'architecture de base (squelette du système)
- Mettre en œuvre cette architecture
- CU critiques, <10 % des besoins
- Faire une planification complète



- Jalon
- « architecture du cycle de vie » = premier prototype qui démontre la validité des choix architecturaux
- Peut-on passer à la construction ?
- besoins, architecture, planning stables ? Risques contrôlés ?

Phase N°2 : Elaboration

- Cette phase reprend les résultats de la phase d'inception et élargit l'appréciation de la **faisabilité** sur la quasi-totalité des cas d'utilisation. Ces cas d'utilisation se retrouvent dans le diagramme des cas d'utilisation qui est ainsi complété.

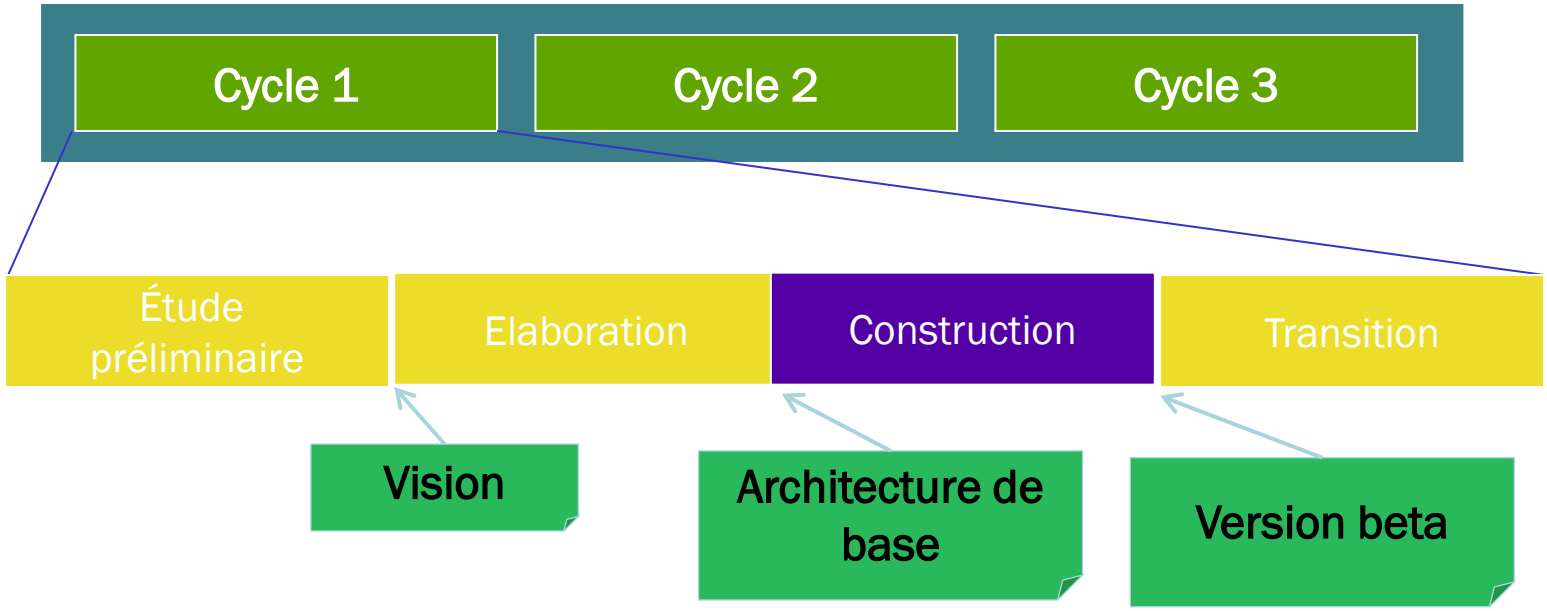
- Cette phase a aussi pour but d'analyser le domaine technique du système à développer afin d'aboutir à une architecture stable. Ainsi, toutes les exigences non recensées dans les cas d'utilisation, comme par exemple les exigences de performances du système, seront prises en compte dans la conception et l'élaboration de l'architecture.

- L'évaluation des risques et l'étude de la rentabilité du projet sont aussi précisées.

- Un **planning** est réalisé pour les phases suivantes du projet en indiquant le nombre d'itérations à réaliser pour les phases de construction.

Phases N°3 : Construction

- Développer par incréments
- l'architecture reste stable malgré des changements mineurs
- le produit contient au final tout ce qui avait été planifié, il reste quelques erreurs



- Jalon
- « capacité opérationnelle initiale » = version béta
- Le produit est-il suffisamment correct pour être installé chez le / un client ?
- Phase la plus coûteuse
- > 50% du cycle
- englobe conception, codage, tests, intégration, etc.

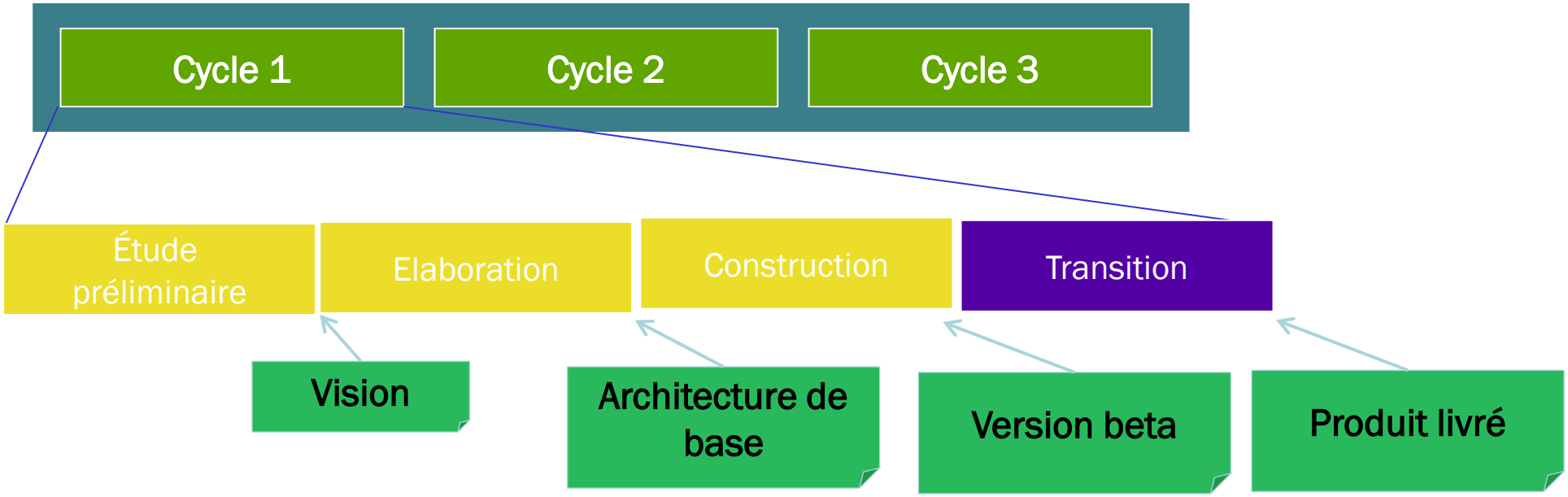
Phases N°3 : Construction

- Cette phase correspond à la **production** d'une première version du produit. Elle est donc fortement centrée sur les activités de conception, d'implémentation et de test.

- En effet, les composants et fonctionnalités non implémentés dans la phase précédente le sont ici.

- Au cours de cette phase, la gestion et le contrôle des ressources ainsi que l'optimisation des coûts représentent les activités essentielles pour aboutir à la réalisation du produit. En parallèle est rédigé le manuel utilisateur de l'application.

Phases N°4 : Transition



Phases N°4 : Transition

- Transition
  - livrer / déployer le produit
  - corriger le reliquat d’erreurs
  - améliorer le produit
  - former les utilisateurs
  - lettre en place l’assistance en ligne

Jalon

« livraison du produit » = produit déployé chez le client
tests suffisants ? Produit satisfaisant ? Manuels prêts ?

Phases N°3 : Transition

- Après les opérations de test menées dans la phase précédente, il s’agit dans cette phase de **livrer le produit** pour une exploitation réelle. C’est ainsi que toutes les actions liées au déploiement sont traitées dans cette phase.

- De plus, des « bêta tests » sont effectués pour valider le nouveau système auprès des utilisateurs.

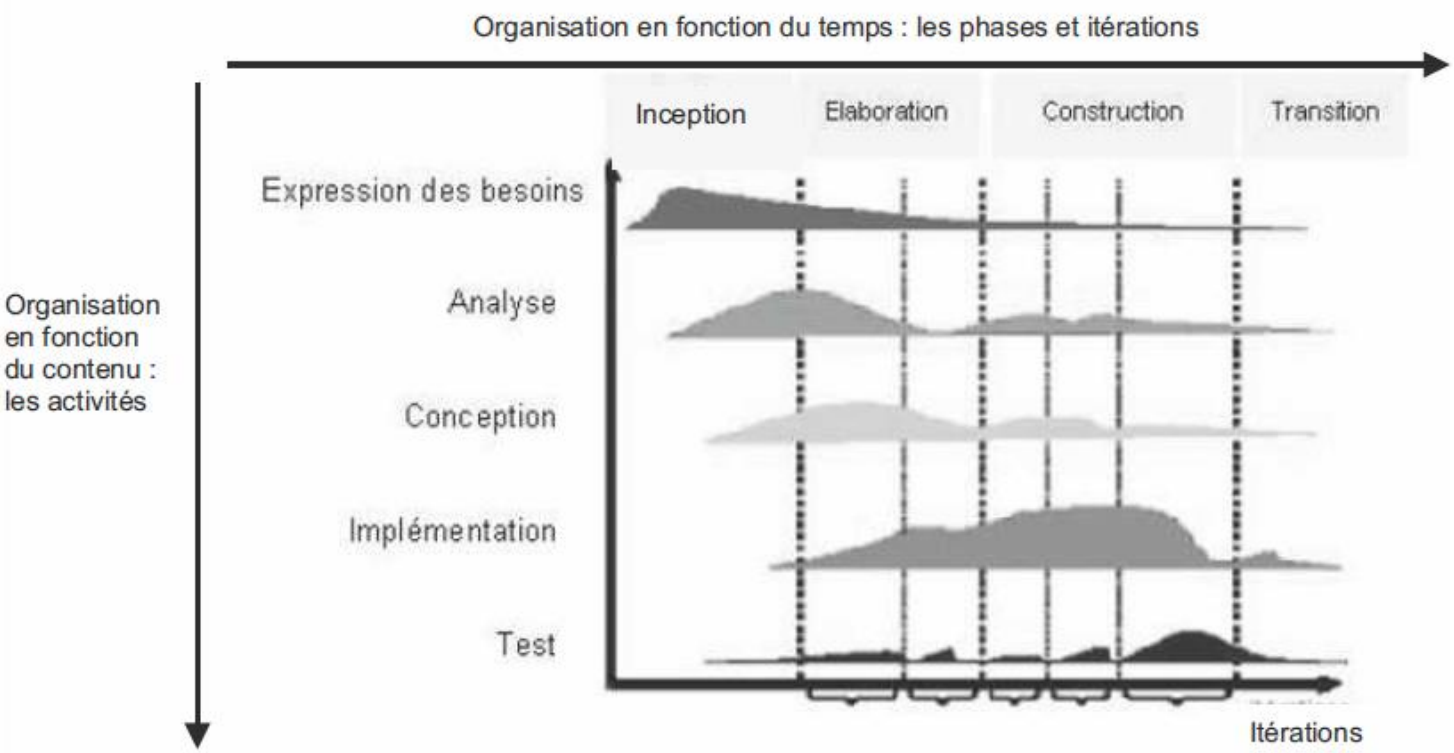
Itération

- Une phase peut-être divisée en **itérations**. Une itération est un circuit complet de développement aboutissant à une livraison (interne ou externe) d’un produit exécutable.

- Ce produit est un sous-ensemble du produit final en cours de développement, qui croît incrémentalement d’itération en itération pour devenir le système final.

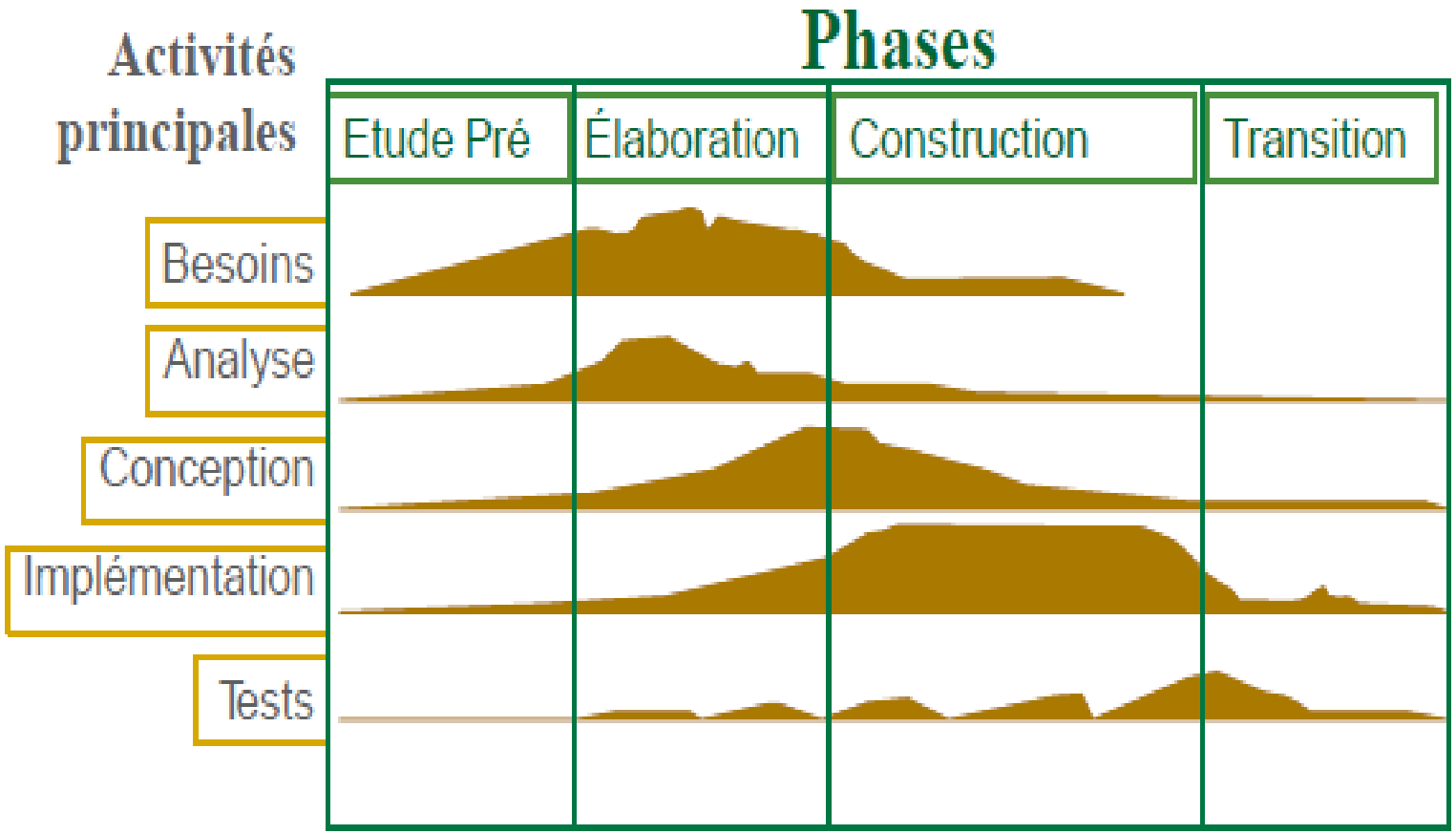
- Chaque itération au sein d’une phase aboutit à une livraison exécutable du système.

Activités du processus
Aspect statique



Activités

- Un cycle met en jeu des activités
- Vue du développement sous l'angle technique (développeur)



Activités de processus

Expression des besoins

UP propose d'appréhender l'**expression des besoins** en se fondant sur une bonne compréhension du domaine concerné pour le système à développer et une modélisation des procédures du système existant.
Ainsi, UP distingue deux types de besoins :

- les besoins fonctionnels,
- les besoins non fonctionnels (techniques) ,

Activités de processus

Conception

La **conception** prend en compte les choix d'architecture technique retenus pour le développement et l'exploitation du système. La conception permet d'étendre la représentation des diagrammes effectuée au niveau de l'analyse en y intégrant les aspects techniques plus proches des préoccupations physiques.

Activités de processus

Implémentation

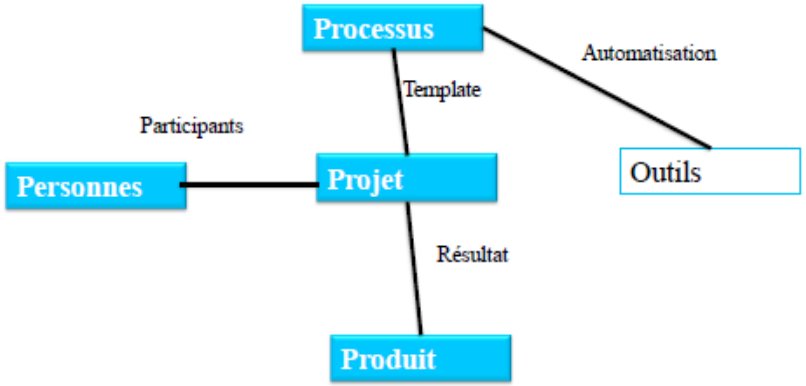
Cette phase correspond à la **production du logiciel** sous forme de composants, de bibliothèques ou de fichiers. Cette phase reste, comme dans toutes les autres méthodes, la plus lourde en charge par rapport à l'ensemble des autres phases (au moins 40 %).

Activités de processus

Les **tests** permettent de vérifier :

- la bonne implémentation de toutes les exigences (fonctionnelles et techniques),
- le fonctionnement correct des interactions entre les objets,
- la bonne intégration de tous les composants dans le logiciel.

4 P du développement logiciel



Personnes: les architectes, développeurs, testeurs, la direction, les utilisateurs, les clients.

Projet: élément d'organisation à travers lequel est géré le développement du logiciel.

Produit: ensemble des artefacts créés au cours du cycle de vie du projet.

Processus: offre un cadre générique à la création de projet.